# Deploying Spring Boot Applications to AWS Lambda with API Gateway

## Why AWS Lambda?

Lambda is a serverless compute service that allows you to run code without the need to provision or manage servers. With AWS Lambda, you only pay for the time that you consume. By deploying your Spring Boot appl$^n$ to AWS Lambda, you can offload the burden of server management and focus on development and delivering your applications.

## Getting started with AWS Lambda and Spring Boot

1st step is to create spring boot project

Then include dependencies

AWS - Serverless - Java - Container - SpringBoot3.

Additionally, the aeronbly-zip module is used to package the appl$^n$ as a zip file.

It creates two important files to facilitate smooth deployment of your AWS Lambda function.

StreamLambdaHandler:

1. This class serves as the entry point for AWS Lambda function to route requests to a Spring Boot appl$^n$ using the AWS Lambda proxy integration.
2. It initializes a Spring boot appl$^n$ context with a Lambda env. It handle HTTP requests via Lambda events (like AWS proxy Request and AWS proxy Response).

template.yaml: —

This template simplifies the deployment process by automating the setup of necessary AWS resources and configuration required to host a Spring boot appl$^n$.

Once your spring boot application and REST API setup, you can package and deploy it to AWS Lambda.

mvn clean package;

To create a new lambda function, navigate to the AWS Console and select AWS Lambda. Then click "create function" and choose "Author from scratch" option.

After creating the function you will need to upload the zip file containing your package appln. Click on "upload from" and select zip file.

Next we need to configure the runtime settings and set the Handler for our lambda function. Handler is the entry point for our appln.

Handler:
Com.Javatechie.StreamHandler::handleRequest

To test the lambda function, you can use AWS Management Console. Click on "Test", and create new test event using the "API Gateway AWS proxy" template. Fill the necessary details.

Now test our lambda function up and running

To create API Gateway navigate to the AWS Management Console and select API Gateway click on build then provide a name of your API and choose "Creat API"

To define the resources and method of our API, we will create a new resource by each endpoint of REST API.

## StreamLambdaHandler

```
public class StreamLambdaHandler implements RequestStreamHandler {
    private static SpringBootLambdaContainerHandler<
    AwsProxyRequest, AwsProxyResponse> handler;
    static {

    }

    @Override
    public void handleRequest(InputStream inputStream,
    OutputStream outputStream, Context context)
        throws IOException {
    handler.proxyStream(inputStream, outputStream,
                        context)
    }
}
```

## template.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform AWS::
Description:
Globals:
    Api:
      EndpointConfiguration: REGIONAL

Resources:
    SpringBootLambdaFunction:
      Type:
      Properties:
          Handler:
          Runtime:
          CodeUri:
          MemorySize: 512
          Policies:
          Timeout:
          Events:
              ProxyResource:
              Type:
              Properties
```

Deployment (Aws Fargate)

of ECS Dashboard create cluster

1. create cluster
2. Under the cluster Configuration enter the cluster name.
3. For infrastructure choose Aws fargate (Serverless)
4. Scroll down and click create

Creating a Task definition

1. Click Task definition then click create new task definition
2. Give the task definition a family name
3. and under launch type select Aws fargate
3. Under the Container details, give the Container a name and Repository url.
4. Set the Container port to 5000

Creating Service

1. On the ECS Console, click the cluster and select the cluster you created
2. Under the Service tab, click on create
3. Use the following Configuration

Compute option : Launch type
Launch type : Fargate
platform version : Latest
Appl$^n$ type : Service

4. Select the Service you created
5. Move to the task defn and click on the running task.
6. Copy the IP address and paste in the browser.

# AWS S3 in Spring Boot

## Introduction

AWS S3 (Amazon Simple Storage Systems) is a scalable cloud storage service by AWS. It allows you to store and retrieve any amount of data from anytime in the internet.

## Feature

1. Storage : Container called bucket.
2. Durability and Availability : offers 99.999999999% durability and 99.99% availability
3. Security:
4. Scalability
5. Data Management.
6. Integration
7. Pricing : charges are based on storage used.

1. Dependency:
   Add dependency in pom.xml file.

2. .yaml properties -

```
aws:
    accen:
    secret:
    account-no:
    s3:
        bucket:
        region:
```

3 create a configuration properties data class to read the properties
   @ ConstructorBinding
   @ ConfigurationProperties(prefix="s3", ignoreUnknownFields=f
   to the class

~~4. Load properties in java class.~~

~~4. Load~~

3. Load properties using java class

```
@Data
@Component
@ConfigurationProperties("aws")
public class AwsProperties {
    private String accessKey;
    private String secretKey;
    private String accountId;
    private S3Properties s3;
    @Data
    public static class S3Properties {
        private String bucketName;
        private String region;
    }
}
```

4. Config to create Bean

```
@Configuration
public class AmazonS3Config {
private final AwsProperties awsProperties
    @Bean("S3AsyncClient")
    public S3AsyncClient s3AsyncClient(...) {
        ...
        ...
    @Bean("s3client")
    public S3Client s3Client(...)
        ...
    }
}
```

5. Service class

```
public class AmazonS3Service {
        ...
    }
```

5a. List All buckets

5b List All object of a bucket.

5c. Upload a file to S3

5d. Delete a file from S3

5e. update a file to S3

5f. Multipart upload File to S3

59. Decentralized G File from S3

6. Controller class

# Deploying a SpringBoot Application on AWS EKS

Kubernetes is an open source system for automatic deployment, scaling and management of containerized (usually Docker) apps.

EKS:

Amazon EKS is a service provided by Amazon Web Service (AWS) that simplifies the deployment, management and scaling of containerized apps using Kubernetes.

Flow Steps:

Spring Boot → Docker → AWS ECR → EKS

Initial Setup:

1. Create spring Boot project —
2. Creat Rest Controller —
3. create Docker file —
4. Generate an executable jar file —
   mvn clean install.
5. create and push the Docker image to AWS ECR —

ECR Setup.

Go to Amazon ECR

- Click on Get started. —
- Click on create Repository. —
- Select view push Command. —
- Click Copy URL to Copy image URL. —

6. Next we need to Setup EKS.

There are two ways to create EKS cluster
1) Using eksctl 2) AWS Management Console.

Next we need to Configure Kubectl to use the EKS cluster for that trigger the below Command.

aws eks --region us-east-2 update-kubeconfig
--name JavaTechie-cluster. —

This command updates the kubeconfig file on your local machine to include configuration details.

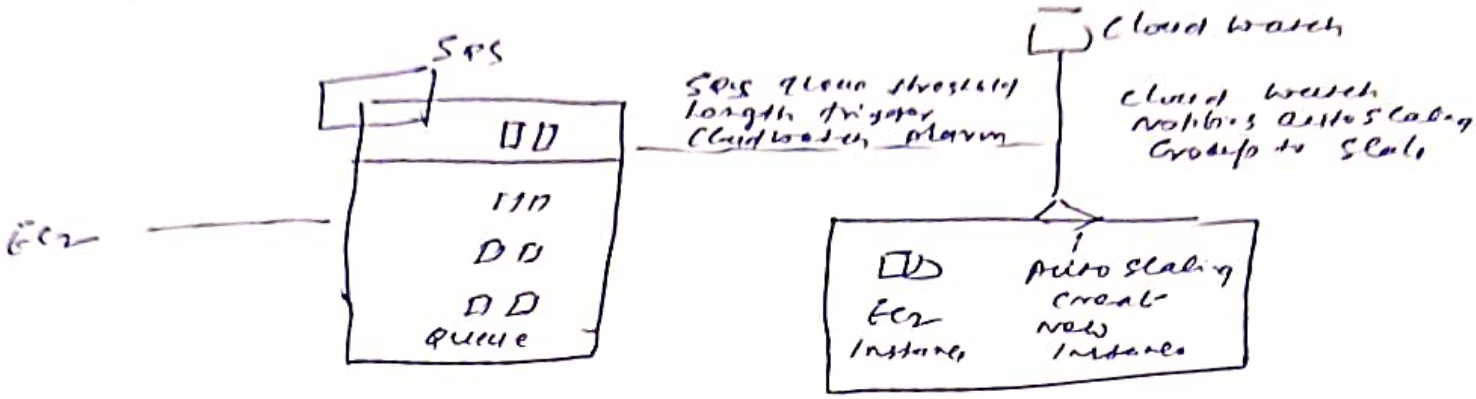
Now that our clusters are up and running.
Next step is to deploy our appln.
to do that we create a Kubernetes file

Next apply manifest file to start deployment and service

kubectl apply -f k8s.yaml

Amazon SQS:

Amazon Simple queue service, How SQS works.




- SQS was the first service available in AWS
- Is a web service that gives you access to a message queue that can be used to store messages while waiting for a computer to process them.
- In this approach one component in the application generates to be consumed by another components where a queue is a temporary repository for messages that are awaiting processing.
- With the help of SQS, you can send, store and receive messages b/w software components at any volume without losing messages.
- Messages can contain up to 256 KB of text in any format such as JSon, xml etc.

Example:

Suppose the user wants to look for a package holiday and wants to look at the best possible flights. The user types the query in the browser, and then hits the EC2 instance. An EC2 instance looks at what the user is looking for", it then puts the message in the queue to the SQS. An EC2 monitors pulls the queue. An EC2 instance continuously pull the queue and looks for the job to do. once it got the job, it then process it. It interrogates the Airline server to get all possible flights. It sends results back to the web server and the web server send back the result to the user. A user then selects the best flight according to his

If we do not have SQS, then what would have happened?

A web browser passes the information to an app server and then the app server queried an airline service. If an app server crashes, then the user loses their queries. One of great thing about SQS is that data is queued in SQS, if the app server crashes, the message in the queue is marked as invisible in timeout interval window. When timeout runs out, the message reappear in the queue, then a new EC2 instance can use this message to perform the job.

Two types of queue.

- Standard queue (default)
- FIFO queue.

Standard queue :- It allows you to have an unlimited number of trans$^n$ per second
- It guarantees that a message is delivered at least once. However sometime, more than one copy of message might be delivered out of order
- It does not guarantees that message are delivered in order

FIFO :- FIFO queue complements the standard queue.
- It guarantees to an order
- Most import feature are FIFO queue and exactly-once processing.
- It does not allow duplicate
- It support message group.
- FIFO queue are limited to 300 trans$^n$ per seconds, but have all capabilities of standard queue.

First Add dependency

in application.properties file

aws.accessKey = your access key
aws.secretKey = your secret key
aws.region = your region
aws.queueName = your queue name.

Create a Configuration class to Configure the AWS SDK and create client for interacting with SQS.

```
@Configuration
Public class SqsConfig {
  @value("${aws.accessKeyId}")
  private String accessKey;
    =
    =
    =
  @Bean
  Public AmazonSQSAsync amazonSQSAsyncClient() {
    BasicAWSCredentials credentials =
        new BasicAWSCredentials(accessKey, SecretKey)
    AmazonSQSAsyncClientBuilder =
        =
        =
  @Bean
  Public QueueMessagingTemplate queueMessagingTemplate(
      AmazonSQSAsync
        =
        =
  }
}
```

Inject the "QueueMessagingTemplate" bean into your class

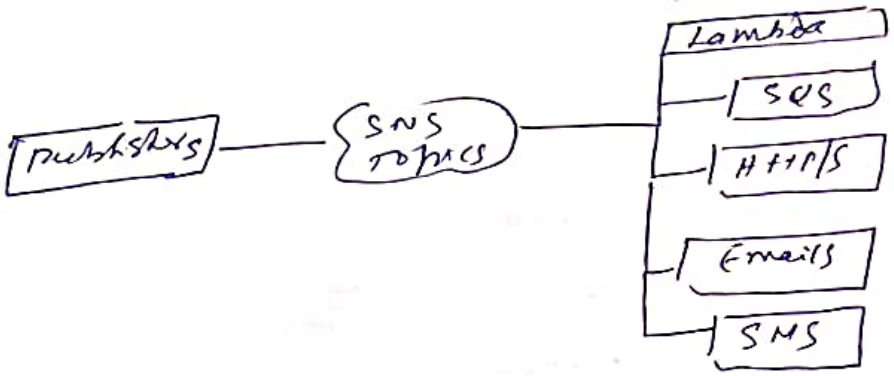
```
@Autowired
private QueueMessagingTemplate messagingTemplate;
```

Now you receive message from the queue.

```
@SQSListener(value = "${sqs.queueName}", - - )
  Public void receiveMessage(String message) {
    =
    =
  }
}
```

AWS SNS

- SNS is fast, flexible, fully managed service
  Push notification service
- It enable developers to send notification to their apps or users by using a variety of message types, such as SMS text messages, email, mobile push notification, and more.
- With SNS, you can publish messages to multiple subscribers or endpoints, and you can also filters the messages based on attributes such message content or user preferences.
- Messages publish to an SNS topics will be delivered to the subscribers immediately.




Topics : • Topics are comm channels,
- You can select topics and inside the topic you can select different subscribers.
- Those selected subscribers will receive the msg which is sent from within the topic.
- Each topic has a unique name.
- Each topic is assigned an AWS ARN once it get created.

Publishers: • publishers are entities that send messages to SNS ~~service~~. Topics.
- Can include AWS services, appl's or custom sources.

Subscribers : Entities that want ... messages from specific topic

. Subscribers can be AWS services, apps or endpoints like (email or SMS)

Protocol: SNS supports various protocols for message delivery, including HTTP, HTTPS, Email, SMS and more.

Filtering :- SNS allows filtering messages based on attributes, so subscribers receive only the relevant content.

## SNS Demonstration

1. Create an Amazon SNS topic
   - Go to Amazon SNS Console
   Click "Create Topic" and give it a name
   - Type at topic standard.
   - Choose the Method Basic

   Now click on Create Topic

2. . Add an Email Subscription to Topic
   . To send an Email, you need to add an Email subscription to topic
   . Click on "Create Subscription" . Select the topic you created in step 1. Click on "Create Subscription" select Email as protocol

3. Confirm the Subscription

4. Publish Messages to the Topic

SNS with Spring Boot

1. Add dependency

2. Configuration file

@Configuration
public class SNSConfiguration {
    @Primary
    @Bean
public AmazonSNSClient amazonSNSClient() {

    }

Key:
Subscribe Email  Rest Controller

@RestController
public class SNSController {
private static TOPICARN = " "
@Autowired
private AmazonSNSClient amazonSNSClient;
    @GetMapping
    for Subscribe

    @GetMapping
    for Publisher.

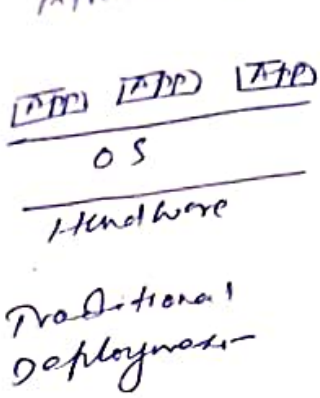
Configuration  application.yaml file.

cloud:
    aws:
        region:
            static: ap-southeast-1
            auto: false
            static
            auto: false.

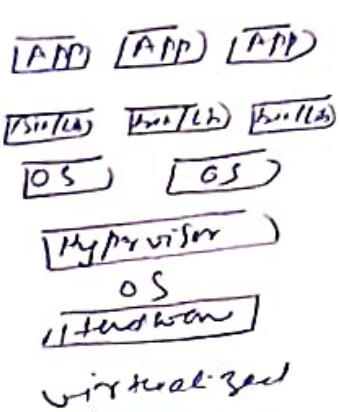
Difference b/w SNS and SQS:

1. SNS supports App to App (A2A) and App to person (A2P) communication, while SQS supports only A2A communication.
2. SNS is Pub/Sub system, while SQS is queuing system. You would typically use SNS to send the same message to multiple consumers via topics. In comparison, in most scenarios, each message in an SQS queue is processed by only one consumer.
3. With SQS, messages are delivered through long polling (Pull) mechanism, while SNS uses a push mechanism to immediately deliver message to subscribed endpoints.
4. SNS is typically used for apps that need realtime notification, while SQS is more suited for message processing use cases.
5. SNS does not persist messages. It delivers them to subscribe that are present, and then delete them. In comparison, SQS can persist the messages (from 1 min to 14 days).

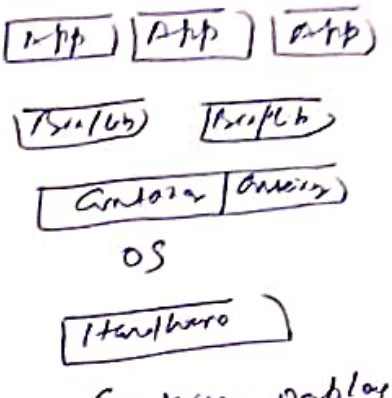
Introduction to Kubernetes: — It is an open source container orchestration tools platform developed by Google. It automate the deployment, scaling and management of containerized apps. Allowing developers to focus on building and running their apps without worrying about the underlying infrastructure.




Traditional Deployment




virtualized Deployment




Container Deployment

Kubernetes consist of two types of resources.

The Control Panel (Master Node): Cordinat the cluster

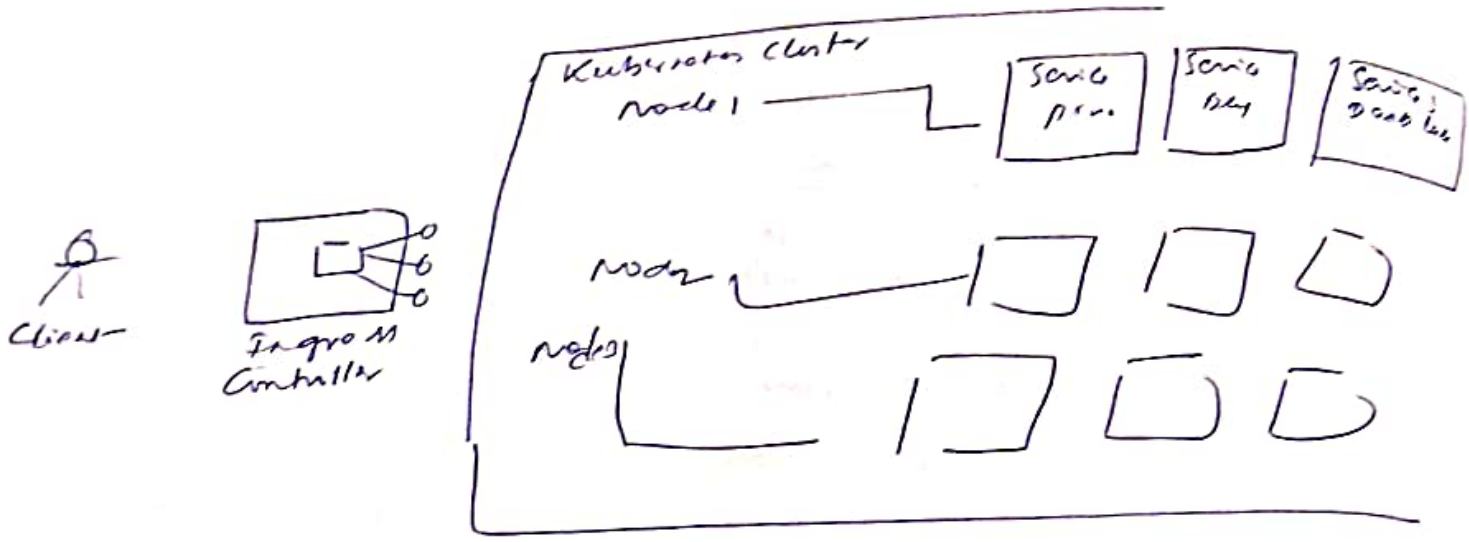
Nodes: Are the workers that run apps. They serve as hosts for Pods that Contain the Containers.

Key Concept:

- Pods: The smallest deployable unit in Kubernetes, Contain of one or more Containers.
- Nodes: The physical or virtual machine that serve a worker machine in Kubernetes cluster. Each node ran a Kubelet, which is an agent for managing the node and Communicating with the Kubernetes Control plane.
- Deployment: Kubernetes resource that manage the life cycle of Pods, ensuring their desired number of replicas are running at all times.
- Service: An abstraction that defines a set of Pods and a policy for accessing them.
- Namespace: - A virtual clusters within a Kubernetes used to divide cluster resources among

- The Control plane is responsible for the overall health of the cluster. The Control plane coordinates all activities in your cluster, such as scheduling apps, maintaining apps' desired state, scaling apps, and rolling out new updates.

Node level components: Run on the kubelet, communicate with the Control plane using the Kubernetes API which the Control plane exposes.




Master node (Control plane)

The Control plane in Kubernetes consists of several key components that manage and control the cluster's state and orchestrate various operations.

The main components.

Kube-api server: The Kubernetes API server acts as the front-end for the Kubernetes Control plane.

Kube-Controller-manager: Is responsible for running various Controller processes that regulate the state of the cluster.

Kube-Scheduler: Responsible for determining where to deploy new Pods within the cluster.

etcd: Is a distributed key-value store that serves as the cluster's persistent storage for configuration data.

Containerized applications run in the cluster hosts are or more pods, which are the smallest deployable units in Kubernetes.

The main components of Kubernetes node are

Kubelet: It is an agent that runs on each node and is responsible for managing the lifecycle of node.

Kube-proxy: It is a network proxy that runs on each node and is responsible for implementing Kubernetes service abstraction.

Container Runtime: It is the software responsible for running containers on the node.

Kubernetes Container Logs: - It is responsible for collecting and forwarding container logs to the cluster's logging infrastructure.

## Deploying your first App^n.

- create a Kubernetes deployment manifest file (deployment.yaml)

```
apiVersion:
kind:
metadata
spec
template
```

- Apply the deployment manifest using the kubectl apply -f deployment.yaml command
- verify that deployment kubectl get pods.

Accessing your Application:
create a Kubernetes service manifest like (service.yaml)

Setting up [illegible]

step 1: Prepare your AWS Env.
1. Create AWS account.
2. Set up IAM Roles.

step 2: Launch EC2 Instances.
1. Choose an AMI
2. Configure Instance Details: Choose an instance type (i.e t2.medium for testing)
3. Add storage: Allocate storage on needed
4. Configure Security Groups: open necessary ports such as 22 for SSH, 6443 for Kubernetes API server and 30000-32767 for node port services.
5. Launch instances.

step 3: Install Docker and Kubernetes.
1. Connect to instance
2. Install Docker.
3. Install Kubernetes Components: Install 'Kubeadm', 'Kubelet', 'KubeCTL'

step 4: Initializing the Kubernetes cluster
1. Initializing the Master node: on the master node, initialize the Kubernetes cluster using "kubeadm"
2. Set up KubeConfig: use kubectl

step: Deploy a Pod network
1. Install a pod network Add-on: Like Flannel

step 6: Join worker node to the cluster.
1. Get Join Command
Kubeadm token create - print-join-command
2. Join worker node:

Step 7. verify the cluster
Kubectl get nodes.

Steps Deploy Application

1 Create Deployment

kubectl create deployment nginx --image=nginx

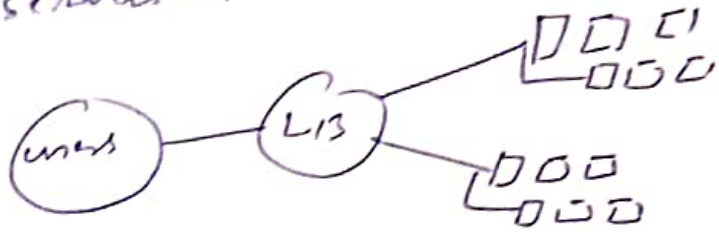
2 Expose Service:

kubectl expose deployment nginx --port=80
--type=NodePort

# Elastic Load Balancing (ELB)

**What is Load Balancing?** LB is a technique that distribute network & appl traffic across several servers. This can improve performance, reliability and availability.

Load balancers work by accepting requests from clients and then routing them to one of the servers in the pool.




Elastic Load Balancing (ELB): Elastic Load balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses in one or more available zones.

It monitor the health of its registered targets, and route traffic only to the healthy targets.

How does ELB works?:-

- Client makes request to your appln.
- The Listeners in your LB receive requests matching the protocol and protocol that you configure.
- The receiving listener evaluates the incoming requests against the rules you specify, and if applicable, routes the request to the appropriate target group.
- Healthy target- is one or more target groups receive traffic based on the LB algorithm and routing rules you specify in the listener.

**The Benefits of Load Balancers**

- Improve performance
- Increased reliability
- Improved availability